

洞察总结：用AI去加速构建AI算力底座，天下武功唯快不破

观点：

- **大模型不只是算力底座的负载，充分用好前沿模型是算力-AI技术栈的竞争力，提升效率，加速数据、模型、软件和硬件的迭代。**【预：frontier模型会自用，更大范围地用于提高内部的生产效率】
- **未来难以预测【不确定】，但以最快速度推出最适合负载的算力底座【确定】是重要的竞争力：**
 - i. **自主coding agent是效率杠杆：使用AI编写更好的代码以加速硬件，提高相同算力的生产力。**【预：25年底覆盖全场景，26年达到人类顶级专家加速绝大多数代码】
 - ii. **大模型加速新芯片、电路和计算系统的设计：**【预：谷歌和英伟达会更大规模使用AI设计新硬件，芯片设计周期可能被压缩到6个月】
 - iii. **通过合成开放环境、自动做算法寻优和迭代，构建高速自我进化的大模型**【预：业界将把benchmark重心转到现实场景】【预：目前还不行，但当创造力，coding能力，长记忆和环境反馈超过奇点时，将出现自我进化】
 - iv. **需要通用的、面向AI开发优化的软硬系统**【预：25年大模型OS化会有更多进展，初见生态雏形】

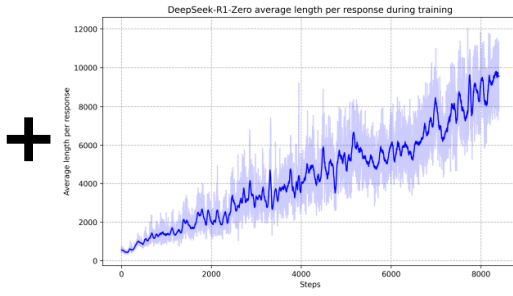
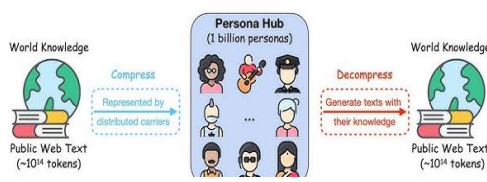
业界利用大模型和AI，不断加速整个技术栈和计算底座

新的scaling law，合成数据+ RL，模型迭代周期缩短

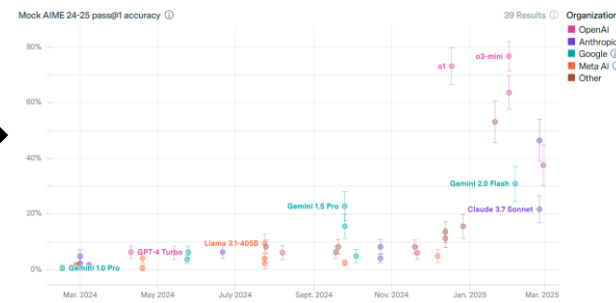
sakana.ai

Can LLMs invent better ways to train LLMs?

Scaling Synthetic Data Creation with 1,000,000,000 Personas



模型能力摸高和新涌现能力



自动优化软件栈，持续加速硬件，支持负载特征变化

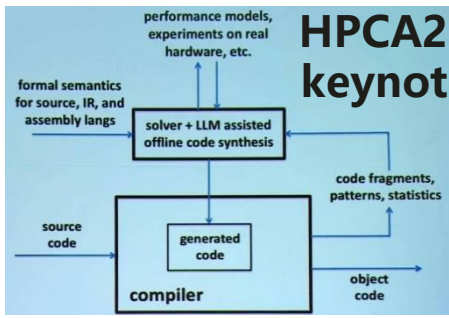
谷歌超过
25%的
新代码都是AI生成



Automating GPU Kernel Generation with DeepSeek-R1 and Inference Time Scaling

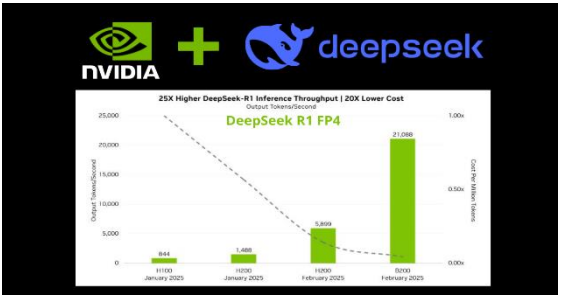
Feb 12, 2025
By Terry Chen, Bing Xu and Kirithi Devlekar

+68 Like Discuss (2)



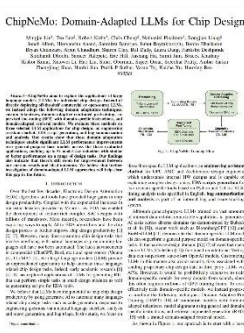
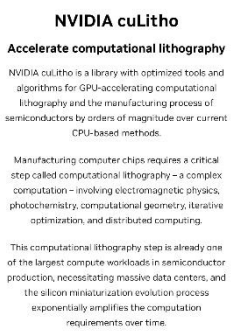
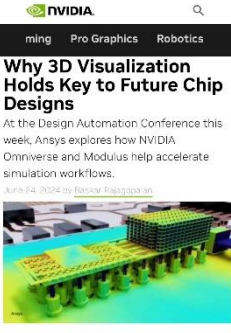
HPCA25 keynote

更高效的kernel和推理软件栈->更高ROI

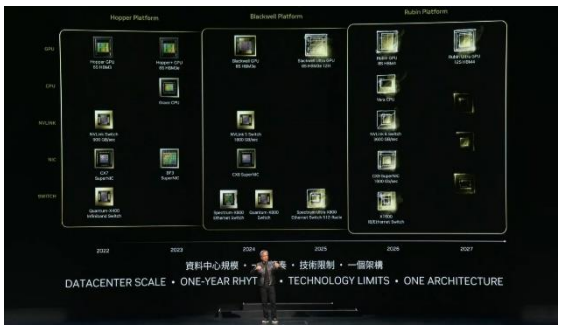


持续缩短新硬件的迭代周期

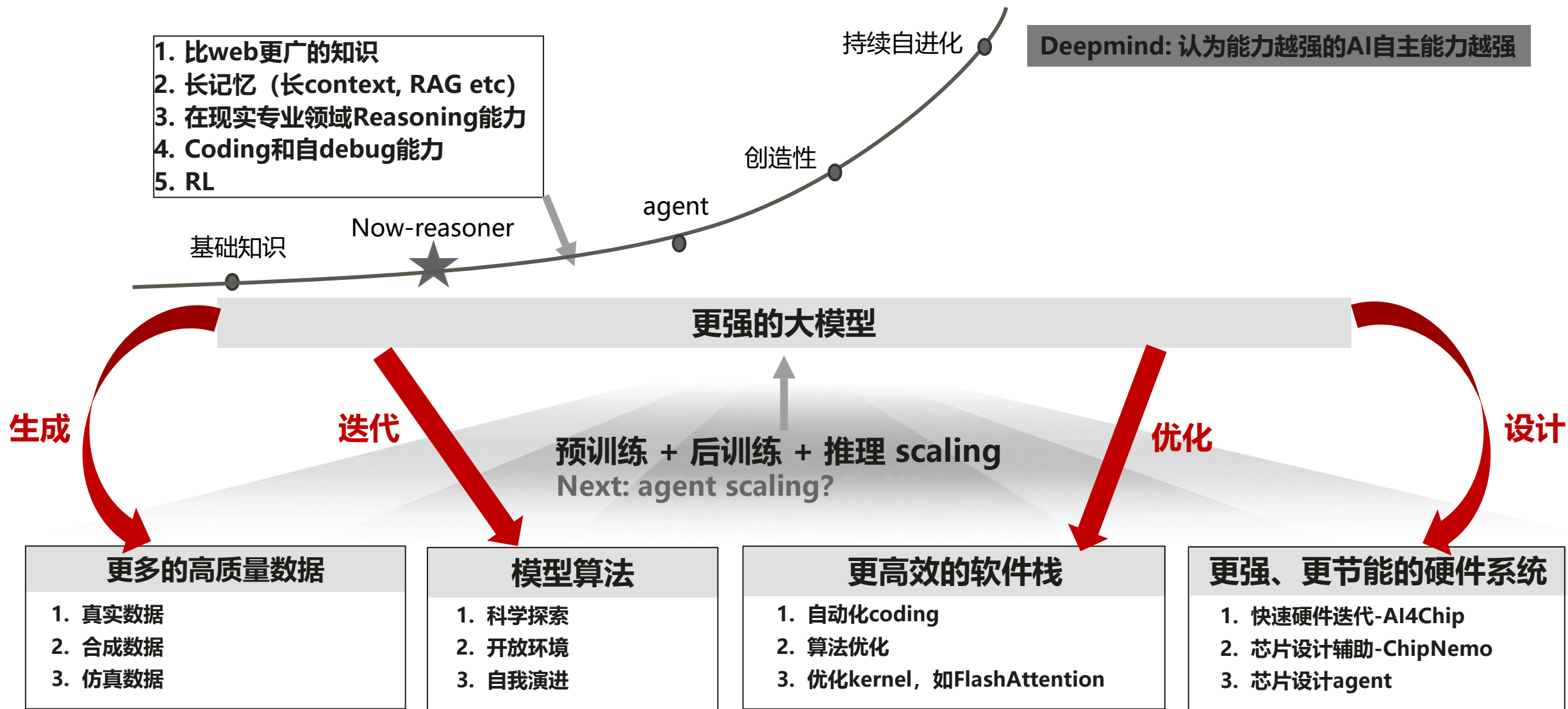
芯片仿真加速+工厂生产流水线加速和优化+光刻过程+计算光刻+芯片设计辅助



两年一代，一年一款



Agent是模型迭代的下一跳: 高质量数据+ 算法自演进+ 更高效软件+ 新硬件迭代



AI加速软件栈

自动化coding+kernel优化

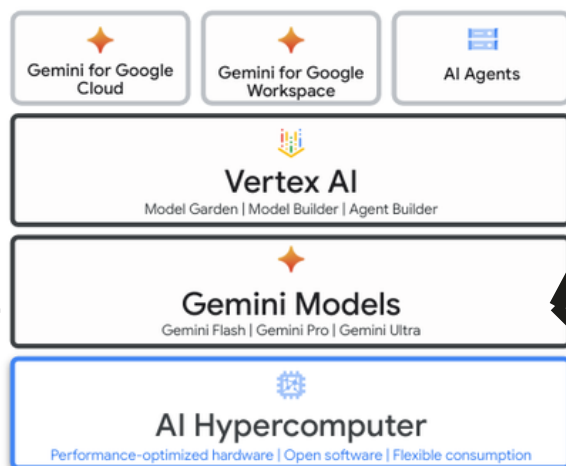
谷歌：以Gemini为中心的統一技术栈，辅助构建AI Hypercomputer，加速frontier模型开发

大模型应用

大模型组合

frontier模型

AI算力底座



Unified technology stack

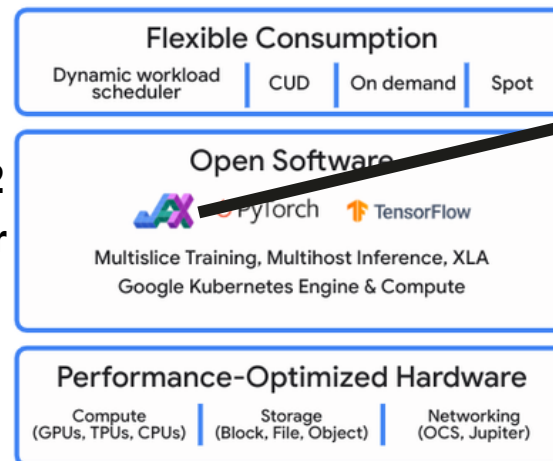
Pathways

Alphacode 2

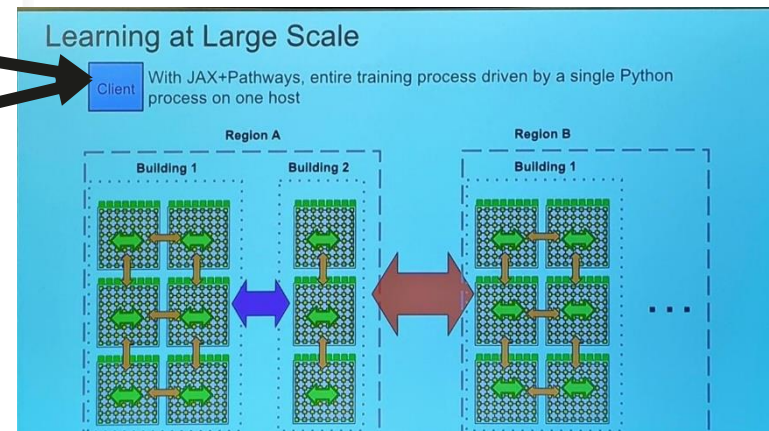
AlphaTensor

AlphaDev

Alphachip



AI Hypercomputer



Learning at Large Scale

Learning for system: 要更大范围利用ML到算力系统的各个部分(OS, 编译器, 内存管理), 现在已经开始发生变化
-Jeff Dean 2024MLSYS

Learning for Systems

Traditional systems (OS, compilers, memory allocators, storage systems, ...) do not make extensive use of machine learning today

This should change (and is starting to change)!

agent自优化代码和算法是其自演进的必备能力

Coding是核心生产力，协助加速开发也占领生态心智

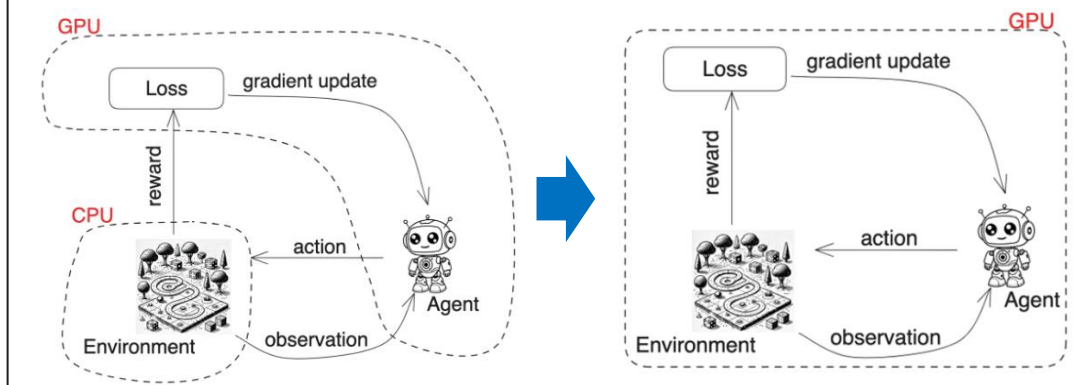
例子1: Gemini code assist (Gemini 2.0作为基模)-25/02/2025

和其他产品集成；在不需要人类反馈下，debug自己生成的代码

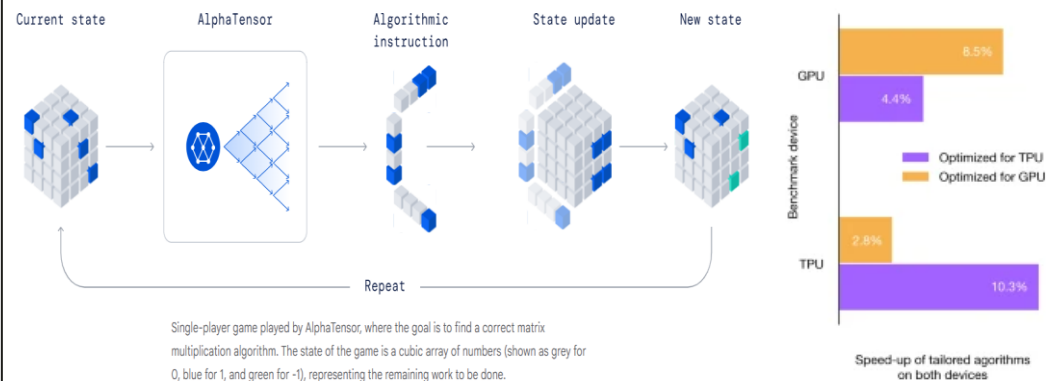
Gemini Code Assist uses large language models (LLMs) from Google. The LLMs are fine-tuned with billions of lines of open source code, security data, and Google Cloud documentation and sample code. These models paired with Gemini Code Assist give developers code completion, code generation, natural language chat, and more, in their IDE, and Google Cloud services including Firebase, Colab Enterprise (Vertex AI), Databases BigQuery, Apigee, and Application Integration.

例子2: 利用LLM coding能力加速RL- Chris Lu, Jacob Forester

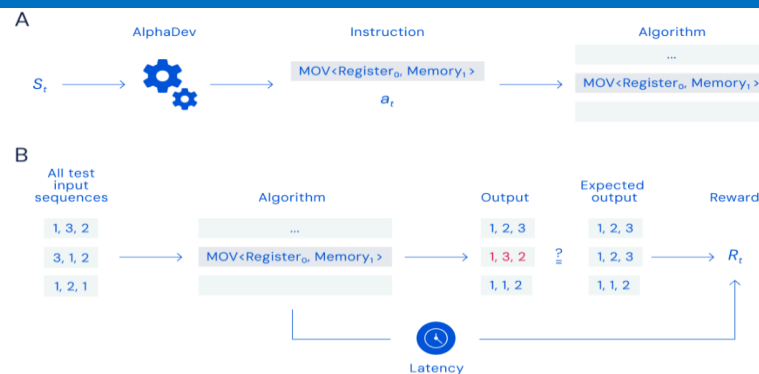
- 将原来跑在CPU的仿真环境迁移到GPU，实现RL加速1000倍
- 利用Jax，更容易地大规模部署在GPU；自动微分/并行等
- LLM用来代码迁移，自动生成和优化RL算法等



AlphaTensor: 发现TPU,GPU上更高效的matmul算法

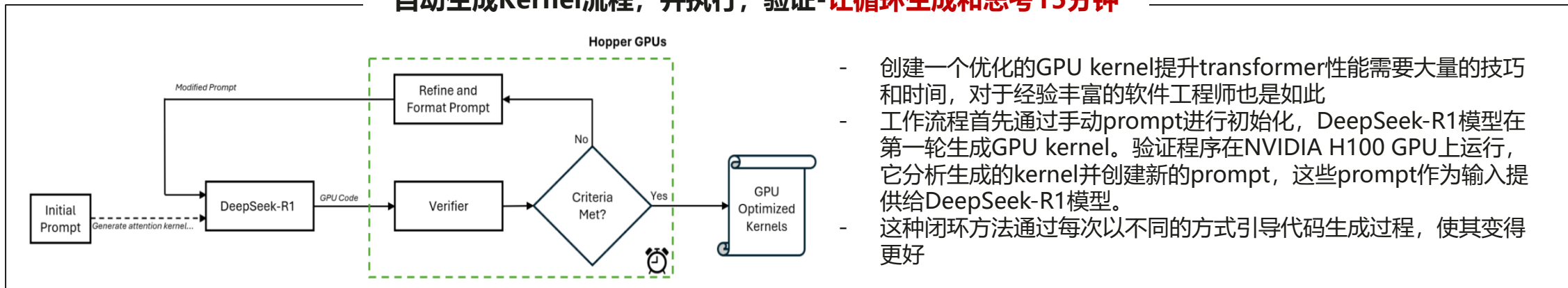


AlphaDev: 更好的排序算法，加快数据处理

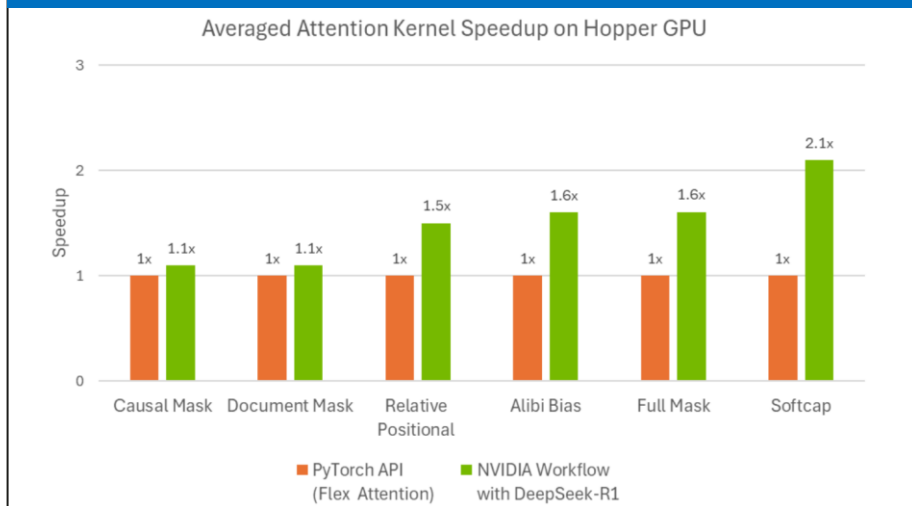


性能加速：英伟达用DeepSeek-R1自动优化GPU Kernel

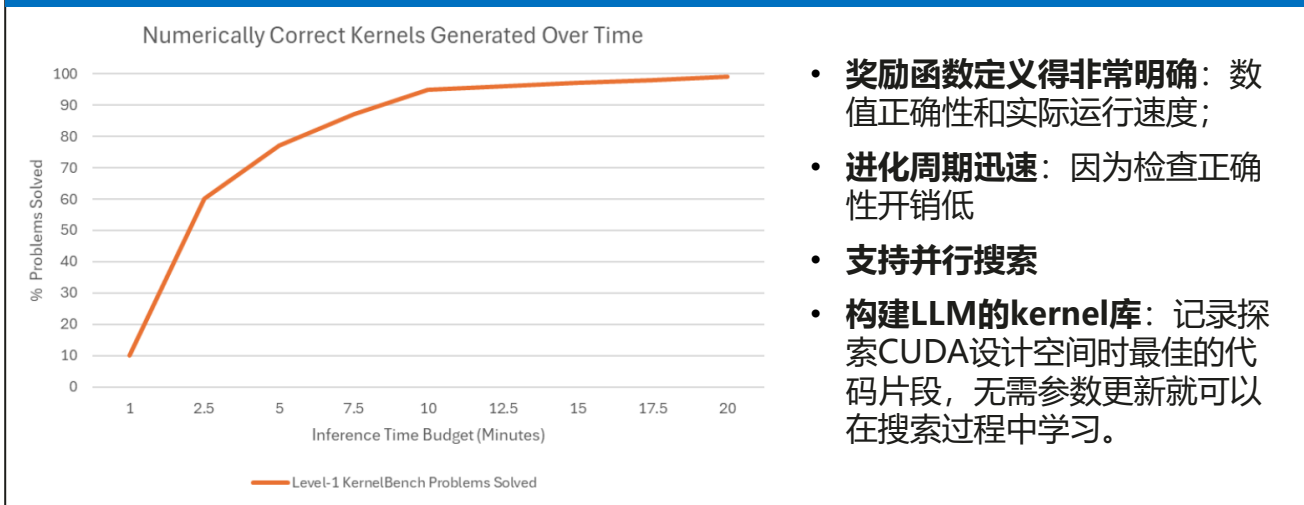
自动生成Kernel流程，并执行，验证-让循环生成和思考15分钟



生成的Kernel在H100上实现加速效果



Test time compute scaling: 20分钟的迭代能完成数值正确的代码



启示：

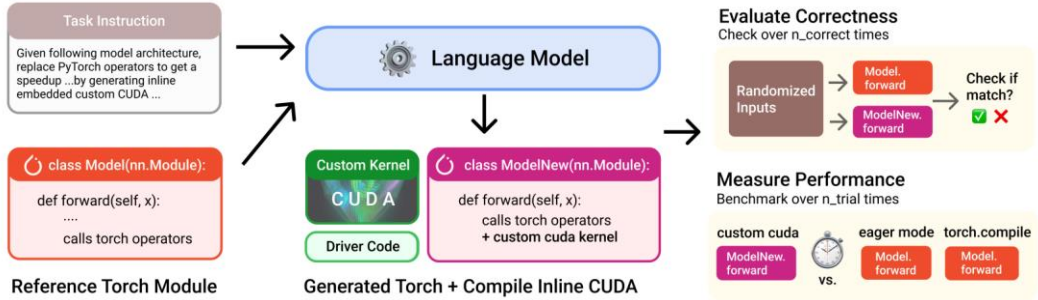
-模型还在进化+TTC可以更高效+大量kernel还没优化

==未来可以生成，更多，更高效的kernel加速GPU，从而让推理速度更快，又能更好TTC scaling，形成循环

KernelBench: LLM生成高效kernel，实现高性能并节约成本和能源

高效kernel很重要但开发耗时且依赖专家->LLM自动优化

LLM生成kernel并评估其质量的框架



- AI工程师在开发kernel时使用丰富的信息集：编译器反馈、分析指标、硬件特定规格和指令集，以及硬件效率技术知识。利用不同汇编语言（如DeepSeek的PTX）和ThunderKittens、Triton的编程工具
- 框架&丰富信息的环境：1. 自动化AI工程师的工作流程；2. 支持多样化的AI算法、编程语言和硬件平台；3. 评估LM生成的性能和功能正确性，能以程序化的方式进行，还可从生成的kernel中捕获分析和执行信息。
- 程序化运行，LM决定优化PyTorch代码的哪些部分以及如何优化它们。

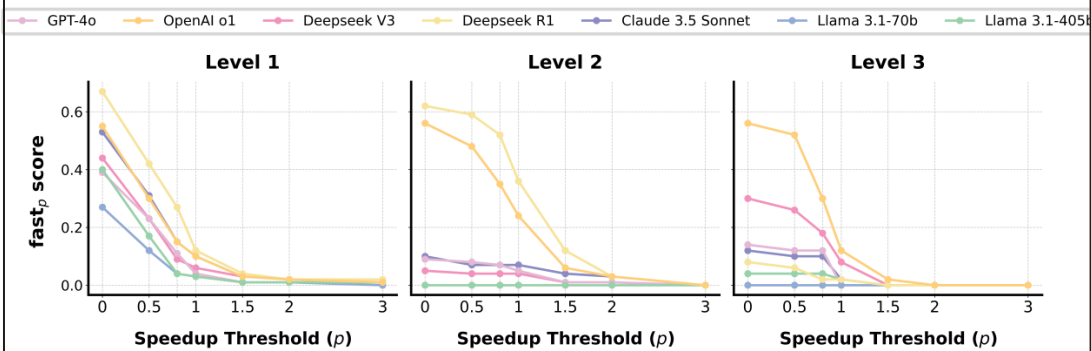
测试LLM在三个级别的AI workload的优化表现 三个级别是：单一操作，连续的操作和端到端架构

KernelBench tests LM optimizations on three levels of AI workloads:

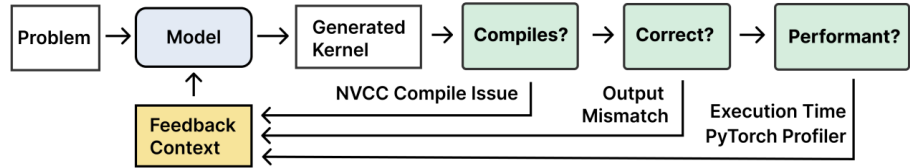
1. **Individual operations:** We include various AI operators, including matrix multiplies, convolutions, activations, norms, and losses. While PyTorch already uses expert-optimized closed-source kernels, making this a potentially challenging baseline, it is valuable if LMs can generate open-source kernels for the operations.
2. **Sequence of operations:** We provide problems that contain 3-6 individual operations together (e.g. a mainloop operator like matmul followed by pointwise operators like ReLU and Bias). This enables evaluating the models' ability to fuse multiple operators.
3. **End-to-end architectures:** We select architectures from popular AI repositories on Github including pytorch, huggingface/transformers, and huggingface/pytorch-image-models. These architectures contain many operations.

LLM可以生成加速kernel，但还有很大提升空间

非LRM生成的大多数kernel不正确，正确的多数没加速效果-》需要LRM



构建feedback context是性能提升的关键



- R1>V3>llama 3.1
- 利用ttc scaling，重复采样可以提高表现
- 通过feedback(生成质量，执行结果，性能profiling)不断调优，可以显著提升性能

Method	Level 1			Level 2			Level 3		
	Llama-3.1 70B	DeepSeek V3	Deepseek R1	Llama-3.1 70B	DeepSeek V3	Deepseek R1	Llama-3.1 70B	Deepseek V3	Deepseek R1
Single Attempt (Baseline)	3%	6%	12%	0%	4%	36%	0%	8%	2%
Repeated Sampling (@10)	5%	11%	N/A	3%	14%	N/A	1%	14%	N/A
Iterative Refinement w G	9%	9%	18%	0%	7%	44%	0%	14%	4%
Iterative Refinement w G+E	5%	13%	41%	5%	5%	62%	8%	22%	12%
Iterative Refinement w G+E+P	7%	19%	43%	4%	6%	72%	2%	14%	18%

METR: 评估生成的优化kernel，促进更高效kernel开发，生态加速度更大斜率

KernelBench不足

- 测试标准挑战不够**：通过检查解决方案的输出是否与参考实现的输出在0.01的绝对容差范围内匹配来**测量解决方案的正确性**。在实际应用中，**kernel还通过端到端模型性能和训练稳定性来测量**（这使得k-best稍微更具挑战性），或者使用更严格的标量容差来测量。
- 基线是未优化的PyTorch代码**：而非高度优化的CUDA代码。这使得KernelBench更易于使用，更能代表小型研究团队的情况，但并不代表大规模应用程序的水平。
- 没有和人类专家对比**：没有将性能与面对相同任务的人类专家进行比较，这使得模型在基准测试上的表现难以解释。
- 不是在GPU集群测试**：都在单个GPU上运行，但并不代表前沿应用的水平，这些应用还包括GPU之间的通信。



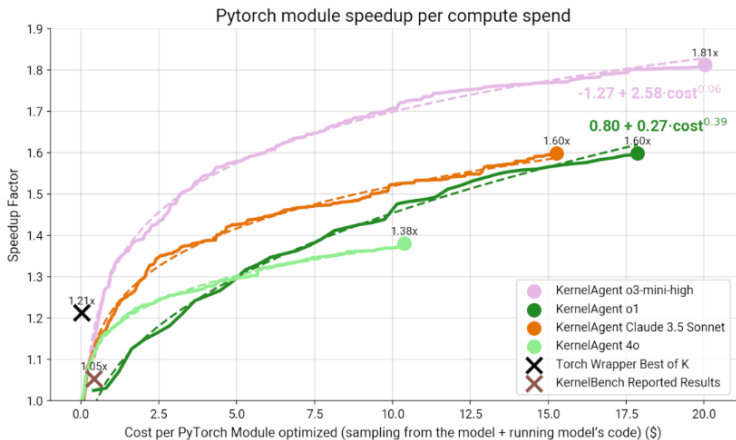
METR重新构建测试集->建议常用kernel都应LRM先优化

- 4个难度等级（对应操难度，kernel数和人类专家预估的开发时间）
- 复杂度逐步提升：单一的pytorch操作->多个pytorch操作->整个模型架构-> SOTA开源库

Level	# problems	Description	Realism	# Kernels per problem	Expert Time Estimate
1	90	Single PyTorch operations, eg CrossEntropyLoss	Realistic, memorizable	1	15 min - 4 hours
2	80	Sequences of 3-6 PyTorch operations, eg Linear->MaxPool3d->ReLU	Unrealistic, novel	1-3	30 min - 10 hours
3	37	Whole architectures from 2010s, eg AlexNet, GRU	Realistic, memorizable	10+	8-100+ hours
5	14	Frontier of open source capabilities and complexity in 2024	Realistic	10+	40-500+ hours

- 在H100上使用o3-mini优化KernelBench任务**平均需要27小时的计算时间才能达到盈亏平衡点**。
- 也就是说，如果一个pytorch模块会被使用超过27小时，那么**应该先用o3-mini去优化，然后运行优化版本会更经济**

inference scaling提升性能，o3-mini-high表现最好



- **业界合作让GPU跑的更快：nebius, modal lab, prime intellect, together ai**
- **有用：算子融合；更高效利用shared和register内存**
- **未来：1. 这些模型定制化微调；2. 探索reasoning更多技巧；3.使用agentic workflow；4. 更高质量的数据库；5. 把这个方法用在其他硬件平台上**

一线专家评价：但是现有技术还不能提高最前沿的kernel

- 和多个前沿AI实验室员工的私下交流：frontier lab在优化每个模型的推理kernel上投入了大量投入【钱+时间】
- 而现有的agent写的kernel和顶级人类专家对比没有加速甚至更慢，所以暂时还无法直接加速frontier lab的工作。

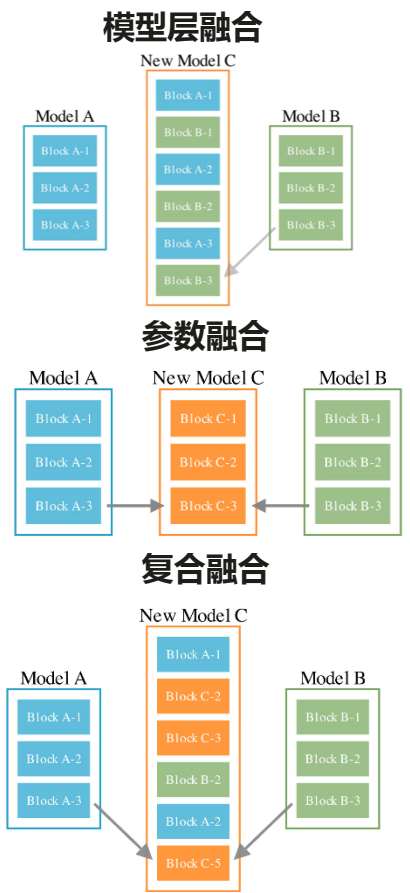
Sakana AI: 自动模型开发+更好的训练方法+更高效的kernel

启示:

- 1. 巨大的搜索空间, 用AI自动化探索更多尝试; 利用了LLM的编程和reasoning能力; 利用了TTC scaling
- 2. 未来需要更长的context利用过程中的profiling和错误信息; 更准的benchmark; 针对任务, 需要更高质量internal thinking的训练数据集

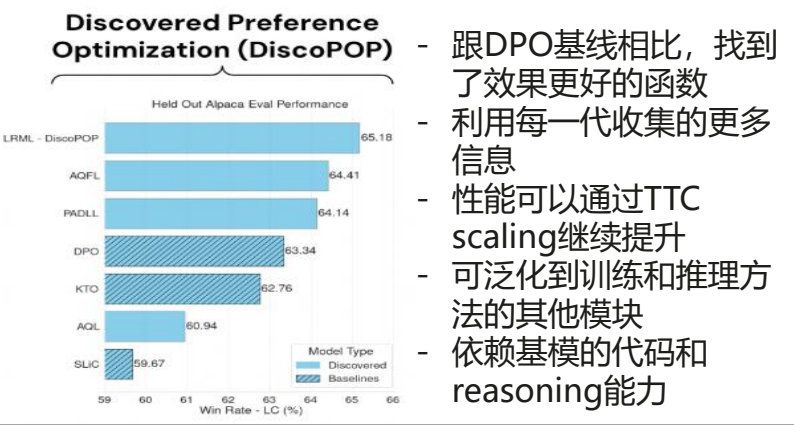
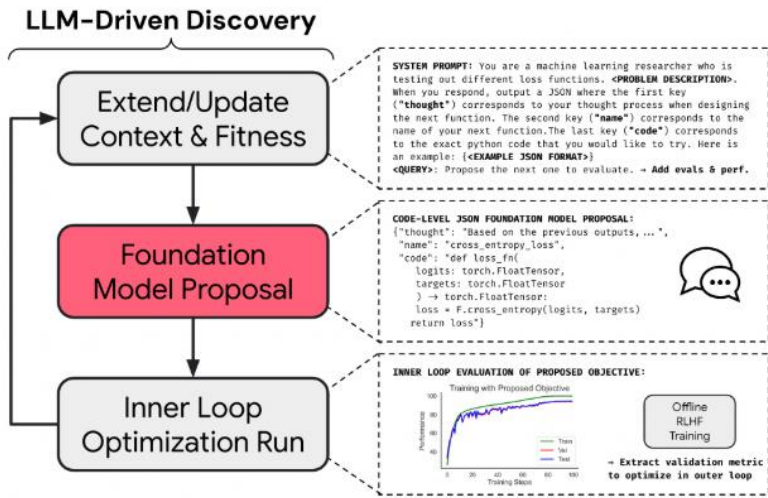
Evolutionary Model merging: 用进化算法自动找到模型融合的最佳组合

- 低成本: 不需要预训练
- 针对下游任务自定义组合合适的模型



LLM-squared: 让LLM迭代更好的偏好优化损失函数

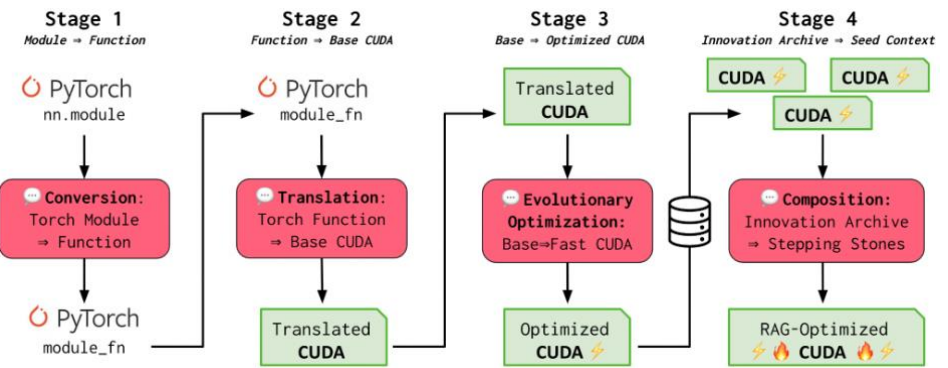
模型更新context->生成新的损失函数->RLHF训练验证
损失函数的效果->将结果反馈作为新的context给回模型



- 跟DPO基线相比, 找到了效果更好的函数
- 利用每一代收集的更多信息
- 性能可以通过TTC scaling继续提升
- 可泛化到训练和推理方法的其他模块
- 依赖基模的代码和reasoning能力

端到端CUDA工程师: 250项CUDA任务中实现平均1.34倍加速

- 将模块torch代码转化为function,再转化为基线CUDA kernel
- LLM基于原有kernel优化, 不断迭代
- 公开了30000由这个agent生成的CUDA kernel



Level 2 [Fused Ops]	# Translated Operations	Improved over torch	Improved over compile	Improved over KernelBench	Torch P50 Speedup	Torch P75 Speedup	Torch Max Speedup
deepseek-v3 [pass@10]	39	14	7	9	0.49	1.08	8.14
sonnet3.5 [pass@10]	52	20	14	13	0.39	1.27	10.81
deepseek-R1 [pass@10]	70	23	16	17	0.37	1.06	59.19
o1-preview [pass@10]	95	42	13	22	1.00	1.01	3.35
o1-high [pass@10]	81	29	17	19	0.87	1.00	4.34
deepseek-v3 [loop@10]	43	15	10	12	0.96	1.11	3.35
sonnet3.5 [loop@10]	64	17	15	11	0.31	1.03	10.16
deepseek-R1 [loop@10]	80	21	18	14	0.26	1.00	112.45
o1-preview [loop@10]	97	45	19	26	1.00	1.04	3.38
o1-high [loop@10]	90	24	13	15	0.51	1.00	6.78
o3-mini-high [loop@10]	88	20	20	17	0.23	0.93	8.64
Union - Max All	98	72	40	41	1.05	1.31	112.45

- 实现kernel优化, 加速明显(发现验证过程有问题, 这些数字不全可信)
- 但这个方法是可行
- 还不能理解底层硬件
- 有的kernel还慢了

AI加速算力硬件设计和优化

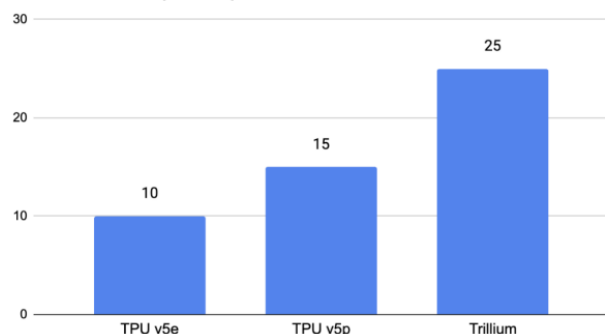
新芯片设计周期缩短到几个月？

谷歌：That chip has sailed-AlphaChip用于TPU设计

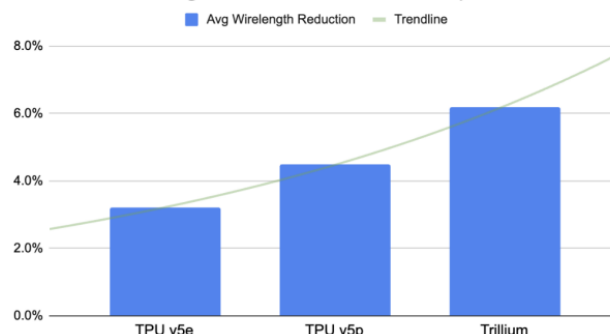
公布时间的间隔缩短，从3年到1年：2018 V3 -> 2021 V4 -> 2023 V5 -> 2024 V6； 预计2026 V7量产

AlphaChip已被部署在三代以上的TPU中。在每一代中，它在更多的模块中被采用，并且以更大的优势超越了人类专家。

Number of AlphaChip-Placed Blocks in TPU Generations



Wirelength Reduction vs. Human Experts

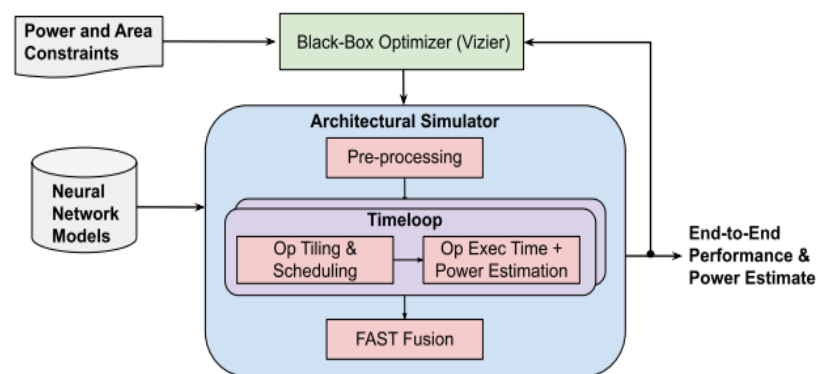


Mar 2024: 7 AlphaChip layouts adopted in Google Axion Processors (ARM-based CPU).

业务负载快速迭代要求芯片加快迭代

	业务占比
TPU v1	MLP/DLRM: 61% RNN: 29%
TPU v3	MLP/DLRM: 27% RNN: 21%; CNN: 24% Transformer: 21%
TPU v4	MLP/DLRM: 24% CNN: 12% Transformer: 57%
TPU v5/v6	transformer

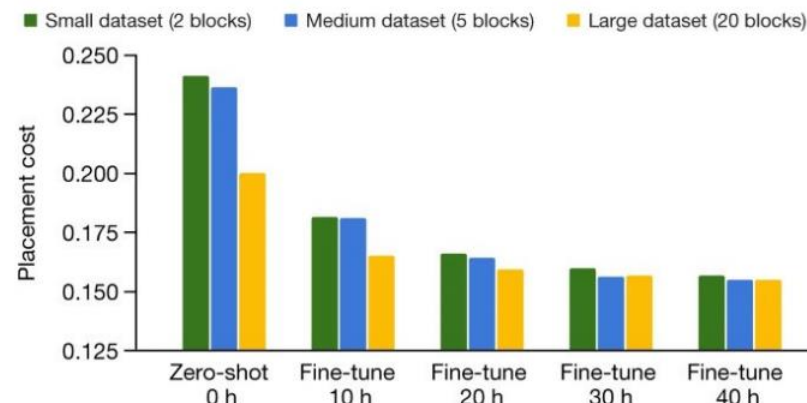
Alphachip+全栈加速器搜索技术: 利用强化学习平均提高Perf/TDP3.7倍



Jeff Dean: AlphaChip是实际生产验证过有效的: “build enough trust for the TPU team to use our layouts; deliver TPU chips and make them as efficient and reliable as possible, and they cannot afford to take unnecessary risks.”

全栈加速器搜索技术, 涵盖了硬件-软件堆栈中的关键设计决策, 包括硬件数据路径、软件调度及编译器等

更大规模的预训练和SFT, 效果越好

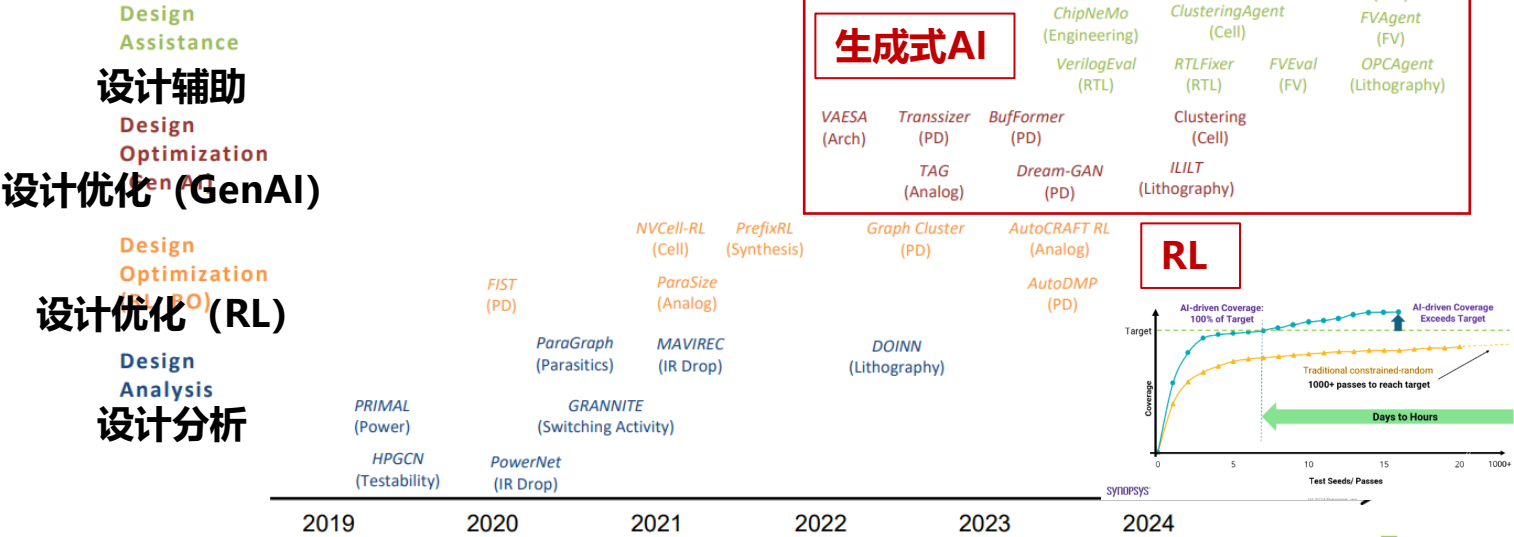


英伟达：用AI设计芯片来训练更好的AI

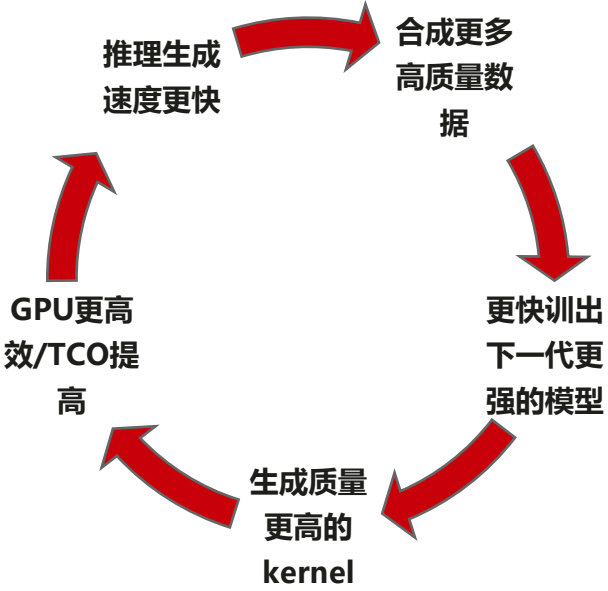
利用AI制造更好的芯片：更大范围使用AI，不断探索利用LLM的能力

AI for Chip Design Research @ NVIDIA

We build AI to build chips for AI!



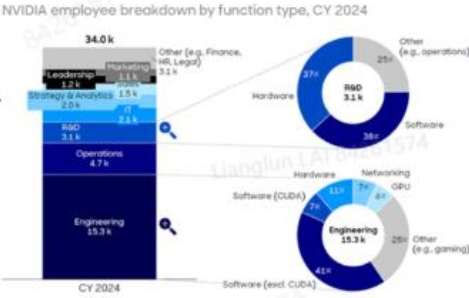
加速飞轮：Kernel优化加速inference scaling



英伟达软件工程师占比高，LLM增强coding和CUDA kernel优化效益高

Side note:

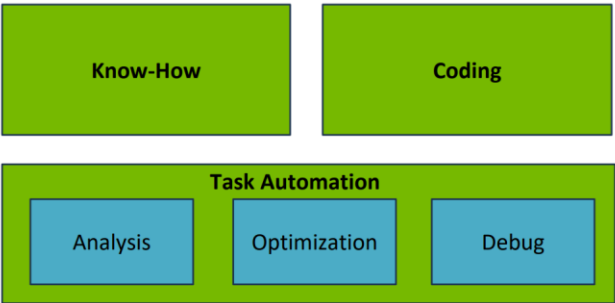
- 工程师约占员工总数的45%，其中约48%专注于软件开发，包括约7%专门从事CUDA相关工作
- 研发部门硬件和软件各占约37%和38%



在设计芯片时，工程师通常面临众多选择和约束。

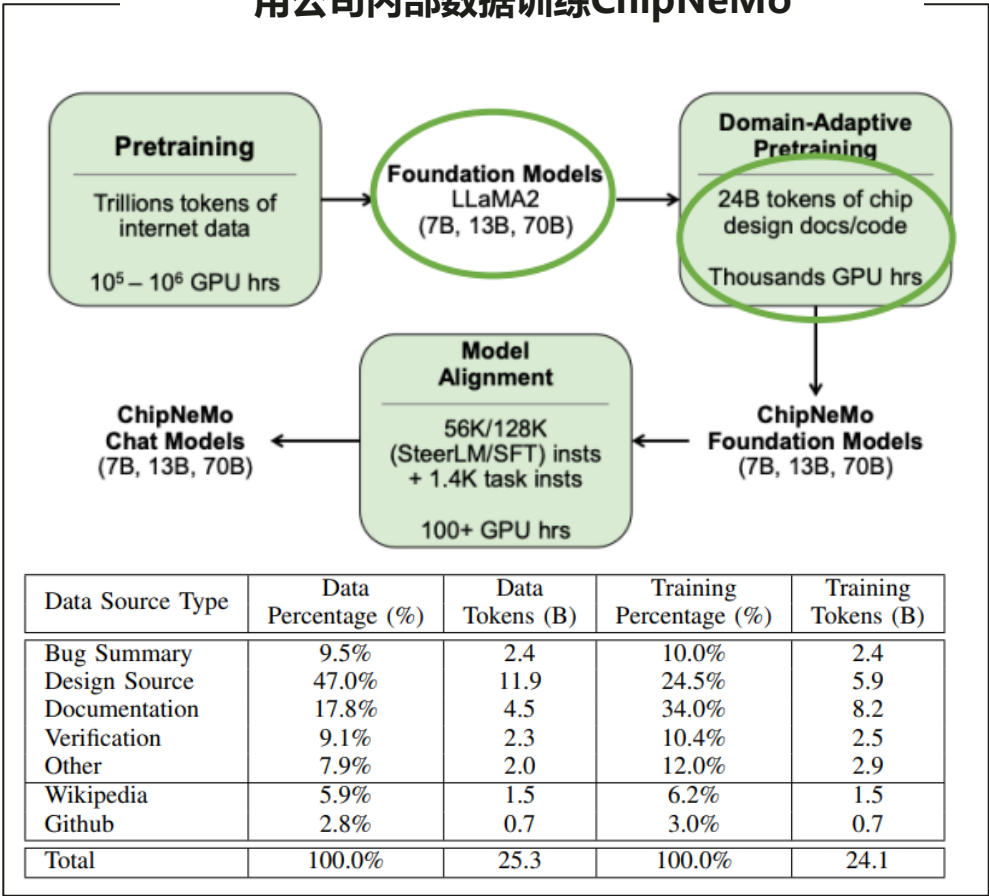
设计空间包括逻辑、面积、单元库、工艺参数、电子设计自动化 (EDA) 工具选择及配置之间的权衡。AI可用于探索极其庞大的设计空间，能够协助设计者探索设计空间，并整合领域特定的知识。

LLM设计辅助：经验+coding+任务自动化

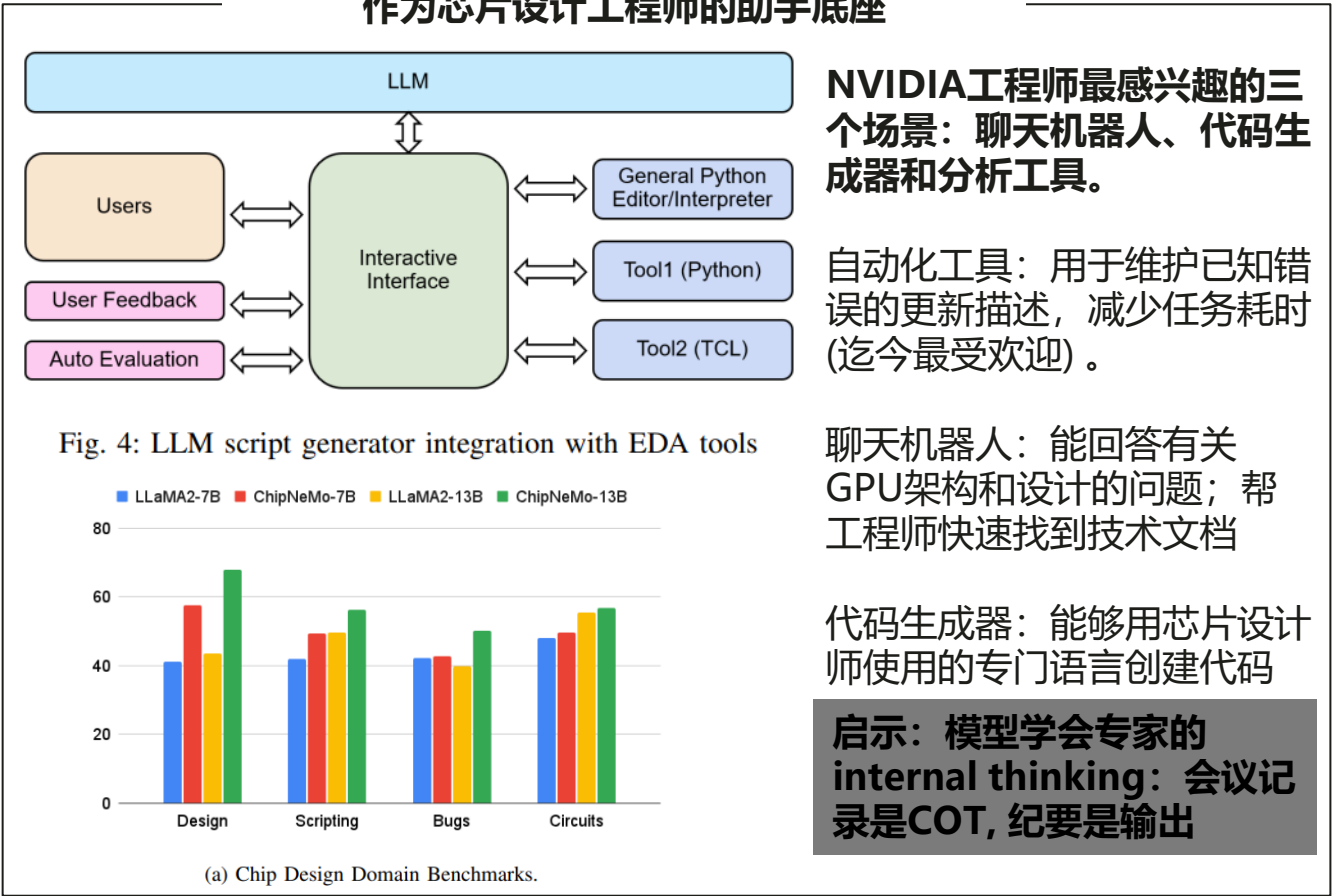


设计辅助：ChipNeMo作为芯片设计领域的通用基座模型，用于多种不同设计任务，提高工程师的工作效率

用公司内部数据训练ChipNeMo



作为芯片设计工程师的助手底座



NVIDIA工程师最感兴趣的三个场景：聊天机器人、代码生成器和分析工具。

自动化工具：用于维护已知错误的更新描述，减少任务耗时（迄今最受欢迎）。

聊天机器人：能回答有关GPU架构和设计的问题；帮工程师快速找到技术文档

代码生成器：能够用芯片设计师使用的专门语言创建代码

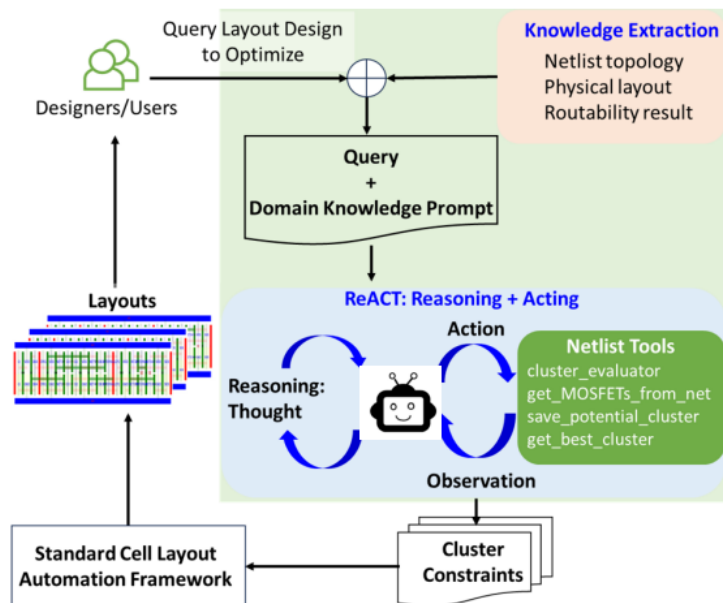
启示：模型学会专家的 internal thinking：会议记录是COT，纪要是输出

- Haoxin Ren：未来长期将应用生成式AI于芯片设计的每个阶段，在整体生产力上获得显著提升，用AI设计更小、更快的电路以及优化大型模块，
- 核心是收集相关的数据和清理这些数据供训练使用；相关代码如软件代码，RTL，验证testbench；自然语言数据库：硬件参数配置，记录文档

设计优化：结合LLM的语言和推理能力和领域知识(设计认知以及特殊工具)，构建了芯片设计agent, 有效提高设计性能和效率

用LLM减少cell placement 5%的面积开销

- 利用LLM的自然语言和推理能力，逐步生成高质量的cluster约束，以优化单元布局的PPA，并使用ReAct调试routability。
- 在2纳米的时序标准单元评测中，不仅成功地将减少cell面积，还能够修复cell布局设计中的routability问题。

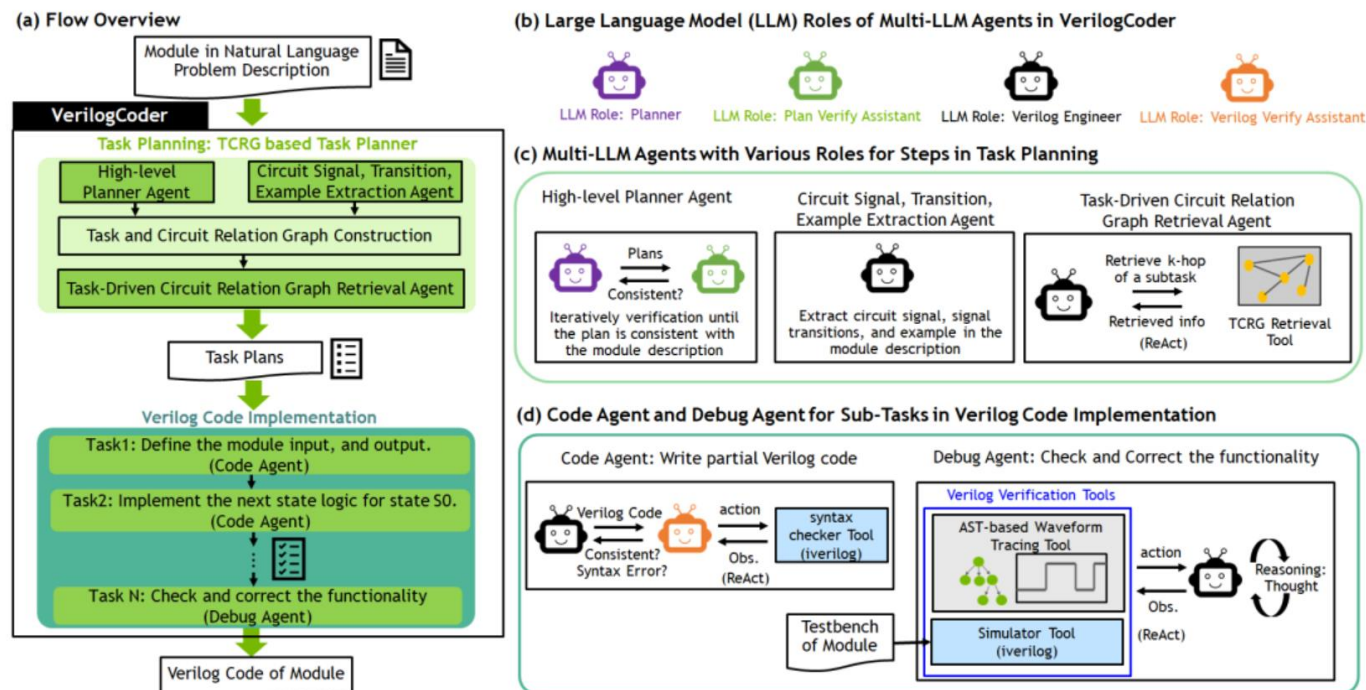


使用预先设计和验证的逻辑单元来构建复杂的电路

C.-T. Ho et al, Large Language Model (LLM) for Standard Cell Layout Design Optimization

VerilogCoder Agent生成正确的Verilog代码正确率94.2%，比SOTA提高33.9%

- 构建了生成Verilog code的AI Agent系统
- 自动调用Verilog工具 (i.e., syntax checker, simulator, and waveform tracer)
- 两级结构，先通过多LLM agent协同+retrival作为任务规划；再通过coding+debug agent执行子任务验证



Verilog HDL（简称Verilog）是一种硬件描述语言，用于数字电路的系统设计。可对算法级、门级、开关级等多种抽象设计层次进行建模

C.-T. Ho et al, VerilogCoder: Autonomous Verilog Coding Agents with Graph-based Planning and Abstract Syntax Tree (AST)-based Waveform Tracing Tool

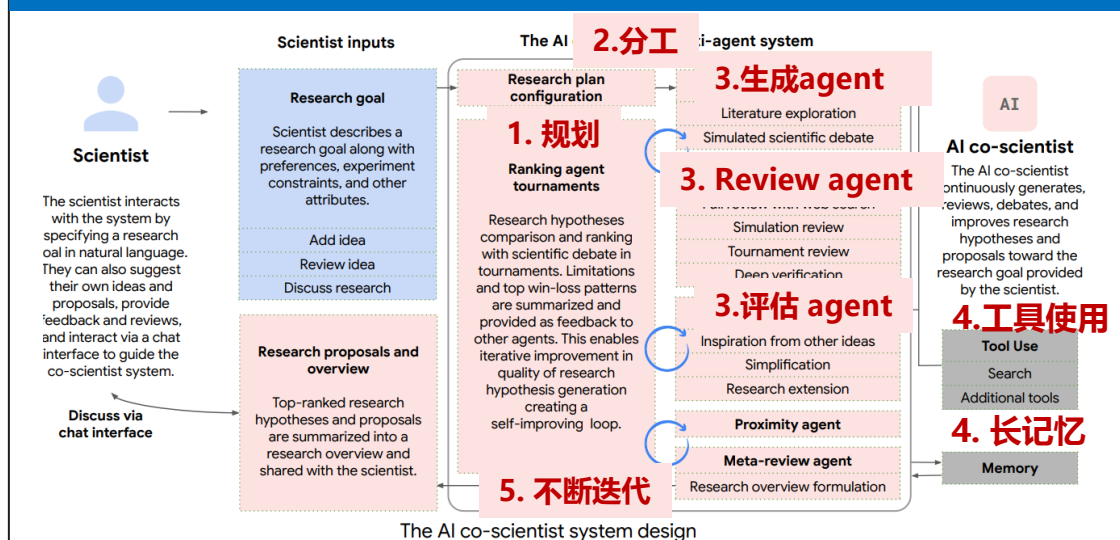
模型自主演进
环境+自主探索+agent算力利用率

科学智能自主探索：谷歌AI co-scientist利用TTC scaling持续self-play，模拟人类科研过程，激发模型提出自我创新

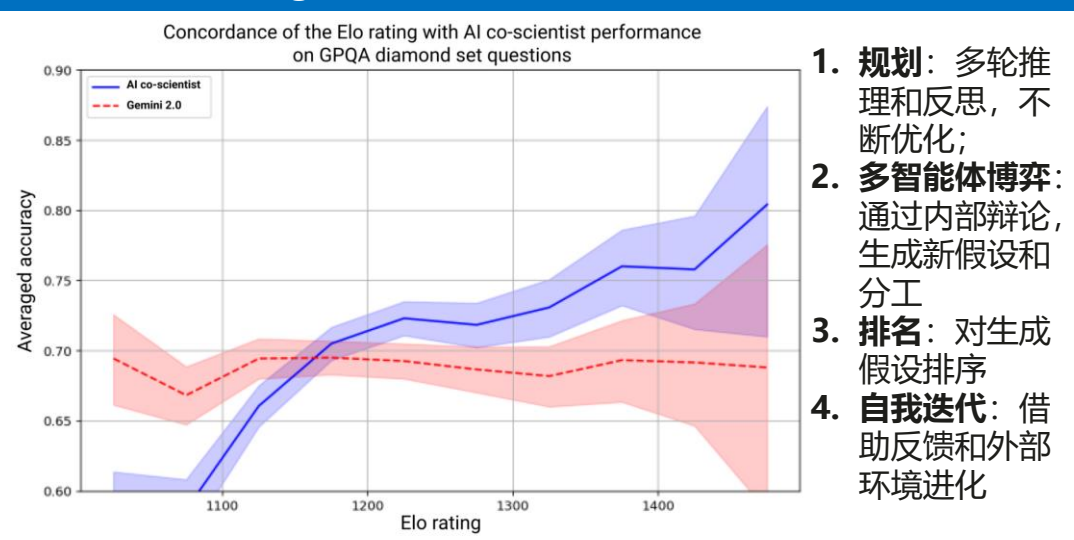
co-scientist基于Gemini 2.0 multi-agent架构，并采用了异步任务执行框架

- **异步任务框架[异步补时延]**：agent在异步、连续和可配置的任务执行框架内作为工作进程运行。一个监督agent管理工作任务队列，将这些进程分配给子任务agent，并分配资源。这种设计使系统能够灵活有效地利用计算资源，并迭代提高其科学推理能力。
- **自然语言界面**：通过自然语言与系统交互并进行监督。定义初始研究目标，对生成的假设（包括解决方案）提供反馈，并总体上指导系统的进展。
- **专业Agent**：根据归纳偏见和科学先验，分解子任务。每个专门的agent都配备了定制的指令提示。
- **上下文记忆**：在长时间范围内实现迭代计算和科学推理，使用持久上下文记忆来存储和检索计算过程中agent和系统的状态。

AI Co scientist多agent系统：在人类给出的研究目标后，连续生成，评估，讨论和提高研究的假设和建议



利用TTC scaling循环增强：通过循环，连续自我演进；利用了模型的reasoning和反思能力

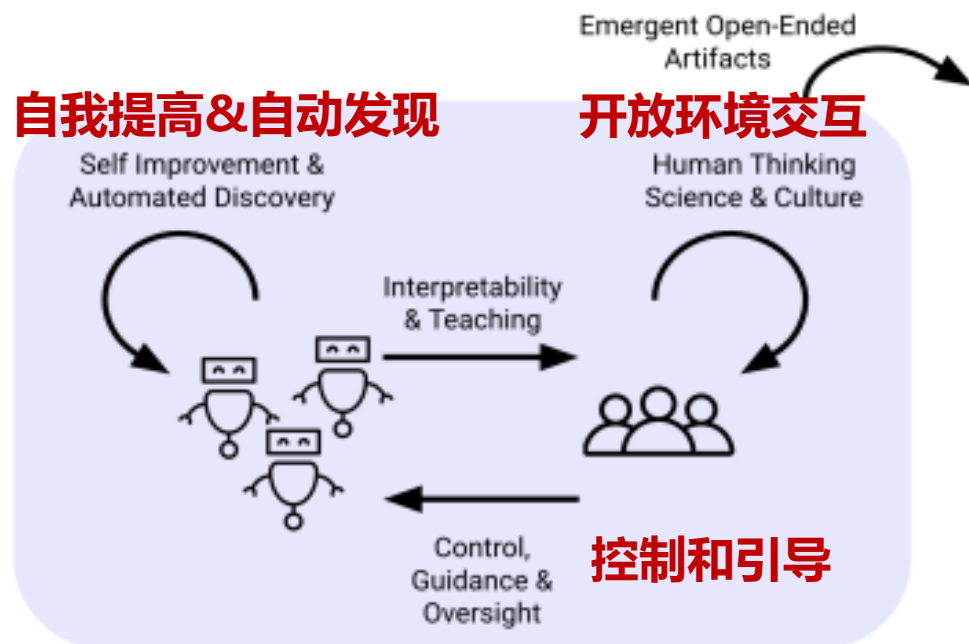


Cong Lu观点: 1. 人处理信息，交互信息的能力是有限的，agent可以scaling；2. open ended任务生成，用agent设计越来越复杂的任务，从而模型可以进化；3. 利用更大的context，Agent有效利用失败的尝试，学会反思；4. 发现并运用新的loss function；5. chains of agents，即是利用了LLM的知识库混合概念，也利用不同agent的特点协作。

GDM OpenEndness伦敦: 开放环境是超人类智能体ASI的必须

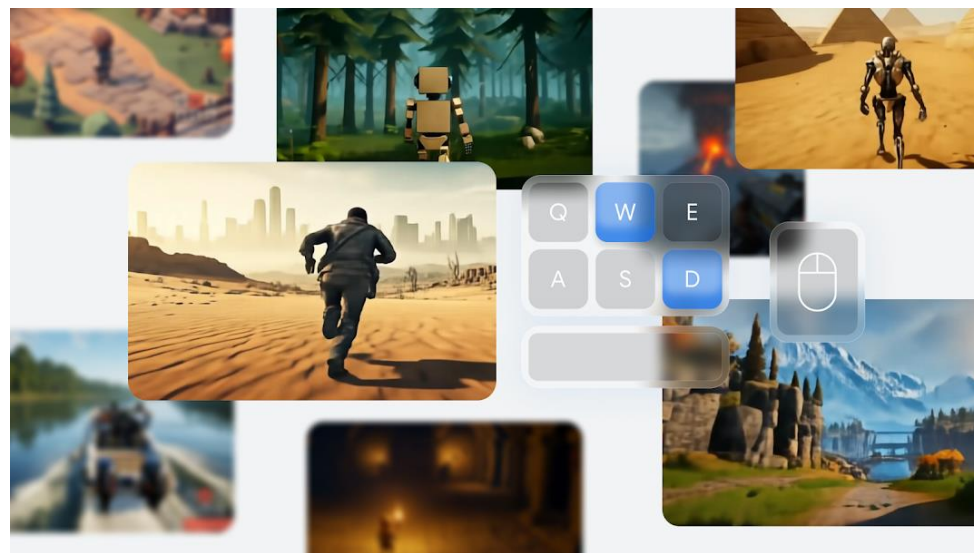
开放性环境为agent自我演进创造条件

- 现实系统非常难: 如何定义正确的目标? 奖励函数? 什么牵引颗粒度
- **Open定义: novel且可学习**
- **保证训练数据是学习所需抽象**
- **多模态数据:** agent将从视频, 声音, 触觉, 传感器等多模态数据去理解环境; 也还可以是动作; 不只是不同模态的输入, 而是不同模态之间相互作用, 相互受益



学会创造, 而不只是做题家

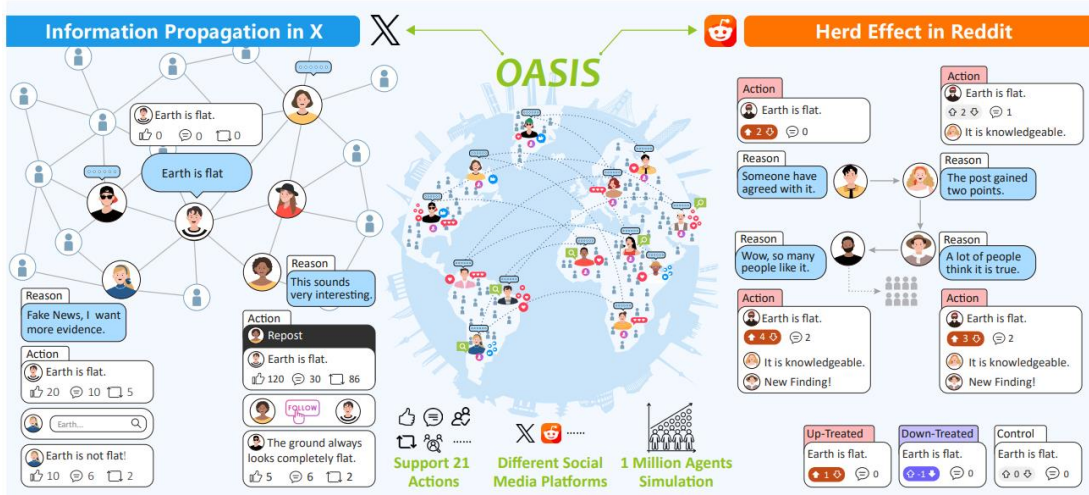
- 通过open-ended play实现通用能力的agent
- **游戏易用高效:** 容易设置奖励函数; 问题有泛化和上下文窗口
- 新的环境中包括没有见过的, 学习到隐空间中的关系; **能生成新的任务**, 自我适应和优化, 完成各种任务
- 在环境中, 学习到如何在世界行动, 并了解行为如何影响世界
- **生成新的知识/能力:** 假设, 洞见, 人类数据所没有的创造性输出
- 例子: Genie 2, SIMA模型



环境：通过游戏，agent群体协作构建模型的交互环境，让模型在交互和反馈中学习和创造

牛津 Camel AI-OASIS: 百万智能体仿真和实验，涌现群体智能

- 让agent在网络上发帖，留言，讨论；
- 随着多智能体规模逐渐扩大，之间的交互变得频繁且多样化，这些群体行为和对应的反馈，进而涌现出群体智能。



微软剑桥-WHAM模型：在3D游戏中利用动作和目标学习

观测游戏世界->理解世界的规则和目标->预测给定行为下的下一帧世界状态



1 观测游戏环境

2 给定动作输入，预测新的世界状态

3 根据游戏(世界)规则，对玩家操作生成打分

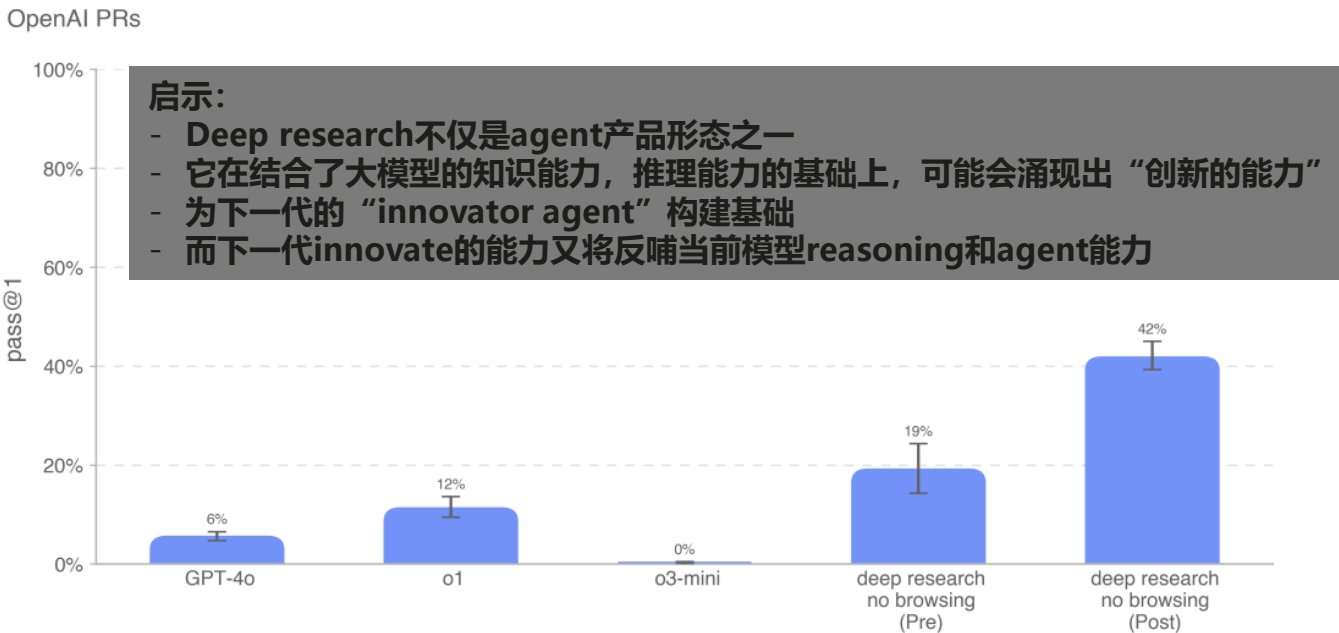
现状：OpenAI判断现有frontier模型的自主能力低

OpenAI持续跟踪模型的autonomy能力，在每个模型的system card中都有测试结果：

- 模型在长周期和代理任务上，自主能力有所提升。
- 模型在自我改进和加速AI研究方面有更大的潜力，但模型在开放式机器学习研究能力的表现仍然不佳
- **自主复制与适应**（Autonomous Replication and Adaptation, ARA）是指模型通过现实世界中获取更多资源来获取智能的过程
- 现有模型的资源获取能力不足，不能充分利用已有资源，如GPU利用率低

测试集和对应的能力	
Evaluation	Capability
ML短任务的能力	OpenAI Research Engineer Interview: Multiple Choice and Coding
	Basic short horizon ML expertise
真实软件工程能力	SWE-bench Verified
资源获取能力	Agentic Tasks
数据和ML相关能力	MLE-Bench
开展ML研究的能力	OpenAI PRs
	SWE-Lancer

O系列推理模型几乎没有开展ML研究的能力；但deep research大幅提升性能



类O模型/agent-硬件之间需要新的软硬协同，提升Agent Flops Utilisation (AFU)?

OpenAI测试Agent/模型未能充分利用给定的资源

- 问题1: 计划小模型的时候用CPU, 大模型用GPU
问题2: OAI尝试的三个不同的agents的解题策略都不能高效的利用所提供的算力和允许的时间
问题3: 有时候会超过内存或存储, 导致任务提前中断

多给一个GPU, agent同样时间内没有提升解题效果

Table 3: Comparing the performance of GPT-4o (AIDE) on different hardware configurations (averaged over 3 seeds, except for *Standard* which used 36 seeds).

Setup	Description	Achieved Medal (%)
CPU-only	Same as <i>Standard</i> but with no GPU provided.	9.1 ± 1.0
Standard	36 vCPUs with 440GB RAM and one 24GB A10 GPU.	8.7 ± 0.5
Extra GPU	Same as <i>Standard</i> but with two 24GB A10 GPUs.	10.2 ± 2.0

agent几乎没有自我思考它们的代码会跑多久

1. Agent/模型不知道pass一次所需的时间和算力
2. 不能理解系统提供的算力
3. 也就无法充分利用系统所有的计算资源

潜在原因1: 训练时没有‘思考如何利用算力; 代码怎么跑的更高效; 代码会需要多少资源;’这一类问题内在思考的训练

潜在原因2: 现有的推理系统还是面向开发人员, 需要人去设计配置局部最优解



为什么关心这个问题? =Agent加速模型开发

- 从Coding agent到**大模型研究员agent**
- Openai测试agent: 1. 能否回答面试问题; 2. 完成大模型开发的代码工程; 3. 能否完成模型开发任务

模型资源利用率高, 不代表agent利用率高

- 有/无人工介入下, agent的资源利用率
- **Agent Flops Utilisation (AFU)?**
- **Agent bandwidth Utilisation (ABU)?**
- **Agent memory Utilisation (AMU)?**
- 异构算力; 分布式算力; 算力资源组合合适?

面向Agent的推理系统设计

模型+agent推理框架需要能够

1. 感知底层的算力资源
2. 感知自身及任务的时间和算力开销
3. 感知资源利用率, 并制定利用算力资源的策略
4. 自动调度资源实现自我优化

启示: 专业的内在思考将成为高质量数据

- 会议记录是COT
- 会议纪要输出
- 模型学会人类专家的内在思考

启示：

- **开发frontier大模型**，并广泛用于日常工作，加速构建算力底座
- **构建更强的推理底座**，更快更便宜地生成token：未来年度生成token数可能是企业的核心竞争力
- **继续提高大模型coding的能力**，如kernel优化加速硬件
- **增强模型自主科学探索的能力**：如领域专家内在思考数据库，会议记录是COT，议题-会议纪要QA对
- **面向AI的软硬件系统**：从AI提高生产力，到AI变成labour；不断替换成更强的模型